

Trabajo Final
Autómatas Finitos
“El As del Barrio”



Fundamentos teóricos de la informática.

grupo: 4

Ayala Emiliano

Curallan Lucas

Pintado José

Vilte Elias

Índice

1. Introducción	2
2. Marco Teórico	2
3. Problemática.....	4
4. Composición del Autómata	5
5. Repositorio	8
6. Bibliografía	8

1. Introducción

Este trabajo busca aplicar la teoría de autómatas finitos (vista en la materia Fundamentos Teóricos de Informática) al modelado de un juego de cartas: el Blackjack (o “21”). La idea es mostrar que las reglas de un juego pueden describirse formalmente como un conjunto de estados y transiciones, y que ese modelo formal puede implementarse directamente en un programa.

Modelamos el juego con dos autómatas complementarios. El primero describe el flujo de una partida (repartir, turno del jugador, turno de la casa, resultado). El segundo describe el puntaje de una mano carta por carta, incluido el doble valor del As. Ese doble valor da lugar naturalmente a un autómata no determinista, que después convertimos en determinista. Además, el autómata del puntaje se presenta como máquina secuencial de Moore, que emite en cada estado una salida para informar al jugador.

2. Marco Teórico

En esta sección definiremos los modelos teóricos a utilizar para modelar la lógica del black jack respetando la notación formal establecida por la cátedra.

2.1 Autómata Finito

Un autómata finito es un modelo matemático abstracto compuesto por una cinta de entrada finita, un cabezal de lectura y un conjunto finito de estados. El modelo lee los símbolos de la cinta uno a uno y según el estado actual sumado al símbolo leído transiciona a un nuevo estado. Si al terminar de procesar toda la cadena de entrada el sistema se encuentra en un estado de aceptación (o final), la palabra es aceptada, en caso contrario es rechazada.

2.2 Autómata Finito Determinista (AFD)

Un autómata finito es determinista si para cada combinación de estado actual y símbolo de entrada existe una única transición posible.

Formalmente un AFD se define mediante la quintupla:

$$AFD = (\Sigma, Q, f, q_0, F)$$

Donde:

- Σ : Alfabeto finito de entrada.
- Q : Conjunto finito y no vacío de estados.
- $f: Q \times \Sigma \rightarrow Q$: Función de transición determinista.
- $q_0 \in Q$: Estado inicial.
- $F \subseteq Q$: Conjunto de estados finales o de aceptación.

2.3 Autómata Finito No Determinista (AFND)

A diferencia del AFD, un AFND permite que una misma combinación de estado y símbolo de entrada tenga múltiples estados como destinos posibles. Esta característica permite evaluar caminos en paralelo o simplificar sistemas con ambigüedad.

Siguiendo la notación de la cátedra, un AFND se define formalmente como

$$AFND = \langle Q, \Sigma, Q_0, F, f \rangle$$

Donde la función/relación de transición mapea hacia el conjunto potencia de estados

$$f: Q \times \Sigma \rightarrow 2^Q$$

y $Q_0 \subseteq Q$ representa el conjunto de estados iniciales.

2.4 Máquinas secuenciales de Estado Finito – Modelo de Moore

Una máquina secuencial extiende el modelo de autómata finito agregando la capacidad de emitir símbolos hacia un alfabeto de salida. En el Modelo de Moore, la salida emitida por la máquina depende exclusivamente del estado al que se transiciona y no de la entrada directa.

Se define formalmente mediante la séxtupla:

$$MEF_{\text{Moore}} = \langle Q, \Sigma_E, \Sigma_S, f, g, q_0 \rangle$$

Donde:

Q : Conjunto finito de estados.

Σ_E : Alfabeto de entrada.

Σ_S : Alfabeto de salida.

$f: Q \times \Sigma_E \rightarrow Q$: Función de transición de estados.

$g: Q \rightarrow \Sigma_S$: Función de salida que asigna un símbolo de salida a cada estado.

$q_0 \in Q$: Estado inicial.

2.5 Gramáticas y Expresiones Regulares

Los lenguajes reconocidos por los autómatas finitos se denominan Lenguajes Regulares, Estos lenguajes pueden ser generados por una gramática regular (GR), la cual se define mediante la cuádrupla

$$G = \langle \Sigma_N, \Sigma_T, P, S \rangle$$

Donde Σ_N es el conjunto de símbolos no terminales (asociados a los estados del autómata), Σ_T es el conjunto de terminales (alfabeto de entrada), P es el conjunto de reglas de producción y S es el axioma inicial. Así mismo, estos lenguajes pueden expresarse de forma concisa mediante Expresiones Regulares (ER) construidas a partir de operaciones de unión (+), concatenación (.) y clausura de Kleene (*)

3. Problemática

3.1. Enunciado

Blackjack es un juego de cartas en el que un jugador se enfrenta a la casa (mediante el denominado crupier). El objetivo es sumar con las cartas un valor lo más cercano posible a 21 sin pasarse, y superar a la casa. Las reglas que modelamos son:

- Se usa un mazo francés. Las cartas del 2 al 10 valen su número; J, Q y K valen 10; el As vale 1 u 11, lo que más le convenga a la mano.
- Al empezar, el jugador y la casa reciben dos cartas cada uno. La segunda carta de la casa está sin revelar.
- En su turno, el jugador puede pedir cartas cuantas veces quiera o plantarse siempre y cuando no supere 21, al hacerlo se excede y pierde.
- Si el jugador suma 21 con sus dos primeras cartas, obtiene blackjack natural y pasa al turno de la casa automáticamente.
- Cuando el jugador se planta u obtiene blackjack natural, es el turno de la casa: La misma está obligada a pedir carta mientras suma menos de 17 y a plantarse con 17 o más.
- Si la casa se pasa de 21, gana el jugador. Si no, gana quien esté más cerca de 21; con igual puntaje hay empate.
- Al terminar una mano, el jugador puede finalizar la partida o jugar otra.

3.2. Alcance y simplificaciones

Para mantener el modelo dentro del alcance de los autómatas finitos y acotarlo el tiempo del proyecto, se establecen las siguientes simplificaciones:

- Modalidad: 1 Jugador contra la casa.
- Mazo: Se asume un mazo continuo/infinito con reemplazo, por lo que la probabilidad de recibir cualquier carta es constante.
- Reglas omitidas: No se contempla la división de pares, doblar apuesta.
- Sin apuestas: El sistema modela el flujo de la partida y el cálculo de puntajes, omitiendo la gestión de fichas o dinero.

3.3. Propuesta

Para solucionar la problemática planteada y no usar un solo autómata complejo optamos por modelar el Blackjack mediante la interacción de dos autómatas complementarios.

- Autómata de flujo de partida (AFD): Se encarga de la lógica general del juego. Modela las transiciones entre el inicio, las decisiones de los jugadores, la jugada del crupier y los estados finales de victoria, derrota o empate.
- Autómata de puntaje de la mano (AFND $\rightarrow$ AFD $\rightarrow$ MEF de Moore): Procesa la secuencia de cartas una a una para calcular el valor total. Inicialmente se define como un AFND para absorber la ambigüedad del As (que puede valer 1 u 11), obteniendo luego el AFD equivalente de 33 estados. A este último le agregamos una función de salida de Moore para alertar el estado de la mano (segura, riesgosa, alta, 21 o pasada)

La casa reutiliza el mismo autómata de puntaje para su turno y ambos autómatas se implementan en un prototipo en Python.

4. Composición del Autómata

AUTÓMATA 1 - CONTROL DEL FLUJO(AFD)

El primer autómata maneja el flujo de la partida

$$AFD_{flujo} = (\Sigma, Q, f, q_0, F)$$

- Σ (Alfabeto de entrada): Representa las acciones del jugador y los eventos de la partida

$$\Sigma = \{r, p, s, x, h, n, v, d, e, c, t\}$$

- r: Repartir cartas iniciales.
- p: Pedir carta.
- s: Plantarse.
- x: Jugador se pasa de 21.
- h: La casa pide carta.
- n: Blackjack natural(21 con las primeras dos cartas)
- v,d,e: Resultados de victoria(jugador) derrota(gana la casa) o empate
- c: Continuar con una nueva mano.
- t: Terminar la partida.

- Q(Conjunto de estados) Modela las fases de la partida:
 - INICIO (q_0): Estado inicial donde la mesa está lista para repartir.
 - REPARTO (q_1): Se repartieron las cartas iniciales; solo desde acá puede leerse n (blackjack natural).
 - TURNO_JUGADOR (q_2): Fase activa donde el jugador elige pedir o plantarse
 - TURNO_CASA (q_3): El jugador se plantó y la casa pide cartas automáticamente.
 - GANA,PIERDE,EMPATE (q_4, q_5, q_6): Estados de resolución de la mano.
 - FIN (q_7): Estado final donde se concluye la partida.
- q_0 (Estado inicial): INICIO
- F (Conjunto de estados finales): FIN
- f (Funcion de transicion): $Q \times \Sigma \rightarrow Q$.

Tabla De transiciones

f_F	r	n	p	s	x	h	v	d	e	c	t
$\rightarrow q_0$	q_1	—	—	—	—	—	—	—	—	—	—
q_1	—	q_3	q_2	q_3	q_5	—	—	—	—	—	—
q_2	—	—	q_2	q_3	q_5	—	—	—	—	—	—
q_3	—	—	—	—	—	q_3	q_4	q_5	q_6	—	—

q ₄	—	—	—	—	—	—	—	—	—	q ₀	q ₇
q ₅	—	—	—	—	—	—	—	—	—	q ₀	q ₇
q ₆	—	—	—	—	—	—	—	—	—	q ₀	q ₇
* q ₇	—	—	—	—	—	—	—	—	—	—	—

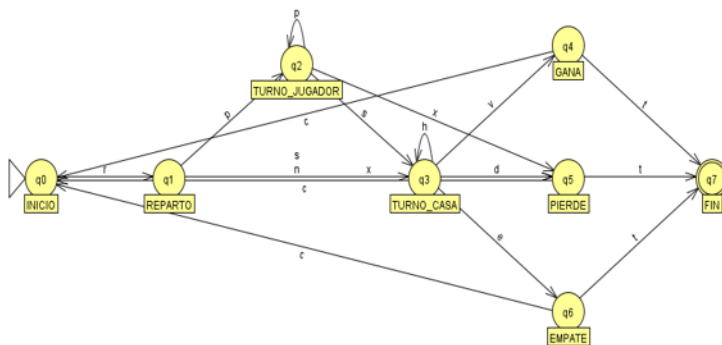
Expresion Regular (ER)

Mano individual (M):

$$M = r \cdot (p \cdot x + (n + p \cdot s) \cdot h^* \cdot (v + d + e))$$

Partida Completa (ER)

$$ER = M \cdot (c \cdot M)^* \cdot t$$



AUTÓMATA 2 - PUNTAJE (AFND → AFD → MEF)

Cada carta es un símbolo de entrada. El 10, la J, la Q y la K valen lo mismo y se agrupan en T; el As vale 1 u 11:

$$\Sigma_E = \{ A, 2, 3, 4, 5, 6, 7, 8, 9, T \}$$

AFND

El estado p_i representa una interpretación de la mano que suma i . Ante una carta distinta del As la interpretación avanza $p_{i+v(c)}$; ante un As se divide en dos ($As = 1$ y $As = 11$) las interpretaciones que superan 21 no tienen transición.

$$AFND_p = \langle Q_N, \Sigma_E, Q_0, F_N, f_N \rangle, \quad Q_N = \{ p_0, \dots, p_{21} \}, \quad Q_0 = \{ p_0 \}, \quad F_N = \{ p_{21} \}$$

$$f_N(p_i, c) = \{ p_{i+v(c)} \} \text{ si } i + v(c) \leq 21 \text{ (}\emptyset \text{ si no), para } c \neq A$$

$$f_N(p_i, A) = \{ p_j / j \in \{ i+1, i+11 \}, j \leq 21 \}$$

Reconoce las manos que pueden valer exactamente 21. por ejemplo con A55 hay dos caminos: con $As = 1$ termina en p_{11} y con $As = 11$ en p_{21} ; como uno llega a un estado final, la palabra se acepta.

AFD equivalente

Para el juego hace falta un AFD: cada estado del AFD representa un conjunto de estados en que podría estar el AFND tras leer las mismas cartas. como a lo sumo un As puede valer 11 sin pasarse, cada conjunto alcanzable tiene a lo sumo dos elementos y se nombran con la terminología del juego:

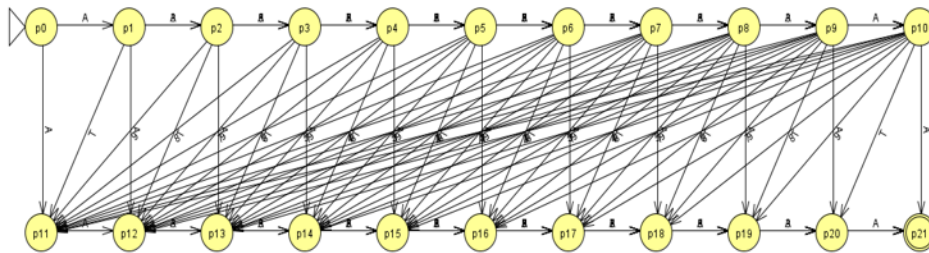
- $D_k = \{ p_k \}$: mano dura de valor k ($D_0, D_2, \dots, D_{21}$: 21 estados).
- $B_k = \{ p_{k-10}, p_k \}$: mano blanda, un As vale 11 ($B_{11}, \dots, B_{21}$: 11 estados).
- $X = \emptyset$: mano pasada, estado sumidero.

$$AFD_p = (\Sigma_E, Q_D, f_D, D_0, F_D), \quad |Q_D| = 33, \quad F_D = \{ D_{21}, B_{21} \}$$

En lugar de una tabla de 33×10 , f_D se resume en reglas (v = valor de la carta $c \neq A$):

Estado actual	Lee A	Lee $c \neq A$
$D_k, k \leq 10$	B_{k+11}	D_{k+v}
$D_k, k \geq 11$	D_{k+1} si $k < 21$; X si $k = 21$	D_{k+v} si $k + v \leq 21$; X si no
B_k	B_{k+1} si $k < 21$; D_{12} si $k = 21$	B_{k+v} si $k + v \leq 21$; D_{k+v-10} si no
X	X	X

Cuando contar el As como 11 pasaría la mano, el As pasa a valer 1 y la mano se vuelve dura (B_{17} con 5 $\rightarrow D_{12}$). Por minimización, como reconocedor solo D_{10} y B_{20} son equivalentes (AFD mínimo de 32 estados); con la salida de Moore se distinguen y los 33 estados son necesarios.



Maquina secuencial de Moore

Al AFD le agregamos una función de salida $g : Q_D \rightarrow \Sigma_S$ que informa el estado de la mano después de cada carta, según su mejor valor v (el mayor que no supera 21):

$$MEF_p = \langle Q_D, \Sigma_E, \Sigma_S, f_D, g, D_0 \rangle, \quad \Sigma_S = \{ S, R, A, V, P \}$$

g	Significado	Estados	Uso
S	Segura: ninguna carta la pasa	$D_0, D_2 \dots D_{11}, B_{11} \dots B_{16}$	La casa pide
R	Riesgosa: dura entre 12 y 16	$D_{12} \dots D_{16}$	La casa pide
A	Alta: entre 17 y 20	$D_{17} \dots D_{20}, B_{17} \dots B_{20}$	La casa se planta

V	Veintiuno	D_{21}, B_{21}	Natural si son 2 cartas
P	Pasada	X	Pierde quien la tiene

Así, la regla de la casa queda expresada directamente con la salida: pide si $g \in \{S, R\}$ y se planta si $g \in \{A, V\}$. Como el 17 blando (B_{17}) tiene salida A, la regla del 17 blando ya está incluida. Por ejemplo, para la mano A65 la máquina emite S A R: el As con el 6 forma un 17 blando, y con el 5 el As pasa a valer 1 y queda un 12 duro.

5. Repositorio

El código del prototipo y los archivos de JFLAP se encuentran en el repositorio: <https://github.com/ayalaEmiliano/el-as-del-barrio>

6. Bibliografía

- Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.
- Cátedra de Fundamentos Teóricos de la Informática, UNPSJB (2026). Apuntes: *Autómatas Finitos (AFD)*, *Autómatas Finitos (AFND)*, *Máquinas Secuenciales*, *Expresiones Regulares*, *Lenguajes Regulares*.
- Rodger, S. H. y Finley, T. W. (2006). *JFLAP: An Interactive Formal Languages and Automata Package*. Jones & Bartlett. <https://www.jflap.org/>
- automathon (librería de Python). <https://pypi.org/project/automathon/>